

Tests for data-generating mechanisms

Stacy Cyrille

2026-06-15

Table of contents

1	Simulate binary variables	3
2	Simulate continuous variables	16
3	Simulate time-to-event data	28
3.1	Aalen additive model	28
3.2	Cox-Aalen model	40

```
library(tidyverse)
library(MASS)
library(knitr)
library(survival)
library(survminer)
library(timereg)
library(gtsummary)

source(here::here("R", "utils.R"))
source(here::here("R", "generate_data.R"))
source(here::here("R", "performance_eval.R"))

options(scipen = 999)

# Set seed for reproducibility
set.seed(2026)
# R <- 1000
R <- 10
```

1 Simulate binary variables

Example 1

We generate a simple example.

- Sample size: $n = 1000$
- Number of DNA pathways: $p = 3$
- Prevalence of DNA pathways: $\pi_1 = 0.1$, $\pi_2 = 0.4$, $\pi_3 = 0.6$
- Correlation matrix:

$$\Sigma = \begin{pmatrix} 1 & 0 & 0.2 \\ 0 & 1 & 0.5 \\ 0.2 & 0.5 & 1 \end{pmatrix}$$

```
n <- 1000
prev <- c(0.1, 0.4, 0.6)

Sigma <- matrix(c(1, 0, 0.2,
                  0, 1, 0.5,
                  0.2, 0.5, 1),
                3, 3)

dna1 <- data.frame(simulate_binary(n, prev, Sigma))
head(dna1)
```

	X1	X2	X3
1	0	1	1
2	0	0	0
3	0	1	1
4	0	0	0
5	0	0	0
6	0	0	0

```
# Check prevalences
colMeans(dna1)
```

	X1	X2	X3
	0.093	0.405	0.603

```
# Check dependence (Fisher's exact tests)
fisher.test(table(dna1$X1, dna1$X2))
```

Fisher's Exact Test for Count Data

```
data: table(dna1$X1, dna1$X2)
p-value = 1
alternative hypothesis: true odds ratio is not equal to 1
95 percent confidence interval:
 0.6397796 1.6015359
sample estimates:
odds ratio
 1.016585
```

```
fisher.test(table(dna1$X1, dna1$X3))
```

Fisher's Exact Test for Count Data

```
data: table(dna1$X1, dna1$X3)
p-value = 0.00005081
alternative hypothesis: true odds ratio is not equal to 1
95 percent confidence interval:
 1.628539 4.962215
sample estimates:
odds ratio
 2.780409
```

```
fisher.test(table(dna1$X2, dna1$X3))
```

Fisher's Exact Test for Count Data

```
data: table(dna1$X2, dna1$X3)
p-value < 0.000000000000000022
alternative hypothesis: true odds ratio is not equal to 1
95 percent confidence interval:
 2.674120 4.802858
sample estimates:
```

odds ratio
3.573428

Example 2

We generate only uncorrelated variables.

- Sample size:

$$n = 1000$$

- Number of DNA pathways:

$$p = 3$$

- Prevalence of DNA pathways: $\pi_1 = 0.1$, $\pi_2 = 0.4$, $\pi_3 = 0.6$
- Correlation matrix:

$$\Sigma = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

```
Sigma0 <- diag(3)
dna0 <- data.frame(simulate_binary(n, prev, Sigma0))

# Check prevalences
colMeans(dna0)
```

```
      X1      X2      X3
0.103 0.397 0.598
```

```
# Check dependence (Fisher's exact tests)
fisher.test(table(dna0$X1, dna0$X2))
```

Fisher's Exact Test for Count Data

```
data:  table(dna0$X1, dna0$X2)
p-value = 0.1375
alternative hypothesis: true odds ratio is not equal to 1
95 percent confidence interval:
 0.8886087 2.1063913
```

```
sample estimates:
odds ratio
1.369885
```

```
fisher.test(table(dna0$X1, dna0$X3))
```

Fisher's Exact Test for Count Data

```
data: table(dna0$X1, dna0$X3)
p-value = 0.8322
alternative hypothesis: true odds ratio is not equal to 1
95 percent confidence interval:
 0.6889528 1.6638722
sample estimates:
odds ratio
1.065647
```

```
fisher.test(table(dna0$X2, dna0$X3))
```

Fisher's Exact Test for Count Data

```
data: table(dna0$X2, dna0$X3)
p-value = 0.01222
alternative hypothesis: true odds ratio is not equal to 1
95 percent confidence interval:
 0.5478133 0.9328525
sample estimates:
odds ratio
0.7149622
```

Example 3

We simulate correlated variables grouped into blocks.

- Sample size: $n = 1000$
- Number of DNA pathways: $p = 20$
- Prevalence of DNA pathways: $\pi \sim \mathcal{U}(0.1, 0.6)$

- Block size: $b = 5$
- Number of blocks: $K = 4$
- Within-block correlations: $\rho_{k1} = 0.6$, $\rho_{k2} = 0.5$, $\rho_{k3} = 0.4$, $\rho_{k4} = 0.1$
- Between-block correlation: $\rho_B = 0.05$

```
p <- 20
```

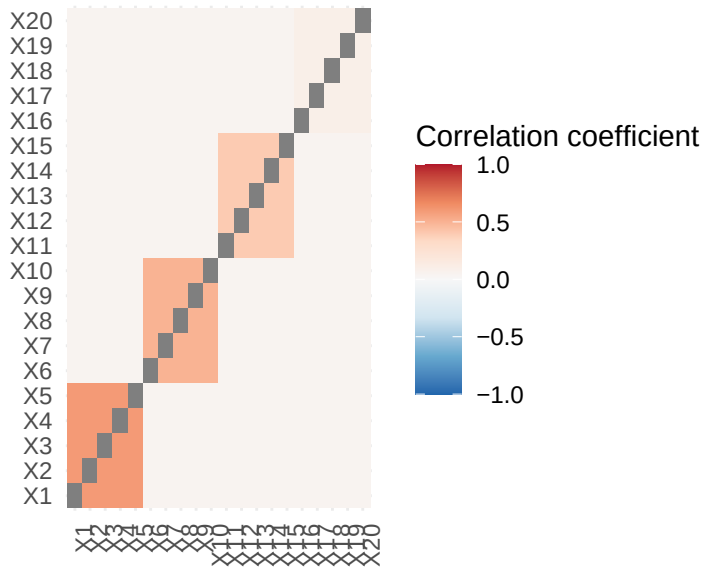
```
# Generate random prevalences
prev <- runif(20, 0.1, 0.6)
prev
```

```
[1] 0.3715163 0.3619613 0.2470627 0.4205293 0.5730484 0.3269650 0.5594099
[8] 0.5307642 0.3007579 0.4936483 0.3841484 0.5842876 0.4502982 0.3408966
[15] 0.2218836 0.2810141 0.1887874 0.2811519 0.4076672 0.5915684
```

```
# Initialize the matrix
Sigma <- generate_block_corr(p,
                             block_size = 5,
                             rho_within = c(0.6, 0.5, 0.4, 0.1),
                             rho_between = 0.05)

# Inspect the matrix
colnames(Sigma) <- paste0("X",1:p)
rownames(Sigma) <- paste0("X",1:p)
Sigma %>%
  as.data.frame()%>%
  tibble::rownames_to_column("var1")%>%
  pivot_longer(-var1, names_to = "var2")%>%
  mutate(value = ifelse(var1==var2, NA, value),
         var1 = factor(var1, levels = paste0("X",1:p)),
         var2 = factor(var2, levels = paste0("X",1:p)))%>%
  {
    ggplot(., aes(var1, var2, fill = value)) +
      geom_tile() +
      scale_fill_distiller(palette = "RdBu", direction = -1,
                          limits = c(-1,1)) +
      theme_minimal() +
      theme(axis.text.x = element_text(angle = 90, hjust = 1)) +
      labs(fill = "Correlation coefficient", x = "", y = "",
           title = "True correlation matrix in the simulation design")
  }
```

True correlation matrix in the simulation des



```
# Simulate correlated data using this matrix
dna_block <- data.frame(simulate_binary(n, prev, Sigma))

# Check marginal prevalences
data.frame(true_prev = prev,
            est_prev = colMeans(dna_block))%>%
  rownames_to_column("variable")%>%
  kable(format = "html", caption = "True and empirical prevalences",
        col.names = c("Variable", "True prevalence", "Estimated prevalence"))
```

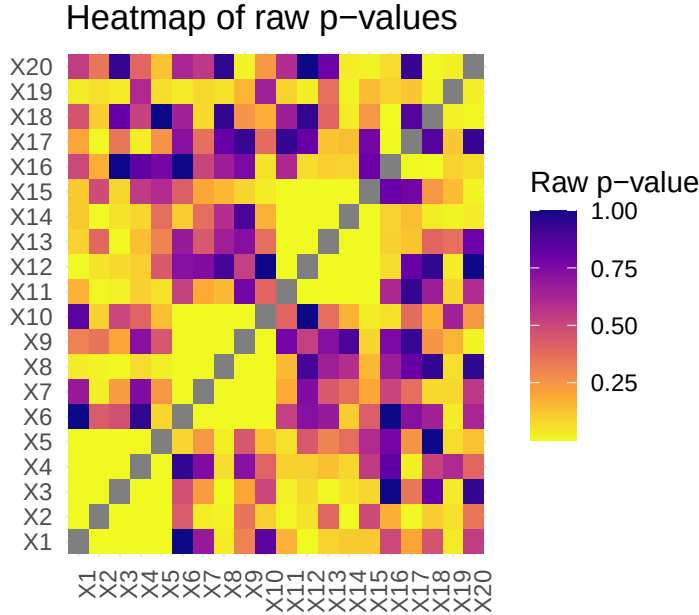
Table 1: True and empirical prevalences

Variable	True prevalence	Estimated prevalence
X1	0.3715163	0.340
X2	0.3619613	0.361
X3	0.2470627	0.243
X4	0.4205293	0.420
X5	0.5730484	0.566
X6	0.3269650	0.311
X7	0.5594099	0.579
X8	0.5307642	0.537
X9	0.3007579	0.308

Variable	True prevalence	Estimated prevalence
X10	0.4936483	0.486
X11	0.3841484	0.396
X12	0.5842876	0.572
X13	0.4502982	0.475
X14	0.3408966	0.320
X15	0.2218836	0.237
X16	0.2810141	0.268
X17	0.1887874	0.192
X18	0.2811519	0.282
X19	0.4076672	0.418
X20	0.5915684	0.598

```
# 20x20 matrix of raw p-values
pval_block <- pairwise_fisher(dna_block, names(dna_block))

# Heatmap of raw p-values
mat_long <- as.data.frame(as.table(pval_block$p.value.matrix))%>%
  mutate(Freq = ifelse(Var1==Var2, NA, Freq))
ggplot(mat_long, aes(Var1, Var2, fill = Freq)) +
  geom_tile() +
  scale_fill_viridis_c(option = "C", direction = -1) +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 90, hjust = 1)) +
  labs(fill = "Raw p-value", x = "", y = "",
       title = "Heatmap of raw p-values")
```



All simulations were repeated across $R = 1000$ replications.

For each replicate, pairwise associations between DNA-based pathways were tested using Fisher's exact test, and the empirical type I error was estimated as the proportion of replications with p-values below the nominal significance level ($\alpha = 0.05$)

We simulate correlated variables grouped into blocks.

- Sample size: $n = 1000$
- Number of DNA pathways: $p = 20$
- Prevalence of DNA pathways: $\pi \sim \mathcal{U}(0.1, 0.6)$
- Block size: $b = 5$
- Number of blocks: $K = 4$
- Within-block correlations: $\rho_{k1} = 0.6$, $\rho_{k2} = 0.5$, $\rho_{k3} = 0.4$, $\rho_{k4} = 0.05$
- Between-block correlation: $\rho_B = 0.01$

```
n <- 1000
p <- 20
prev <- runif(p, 0.1, 0.6) %>%
  set_names(paste0("X", 1:p))
Sigma <- generate_block_corr(p,
```

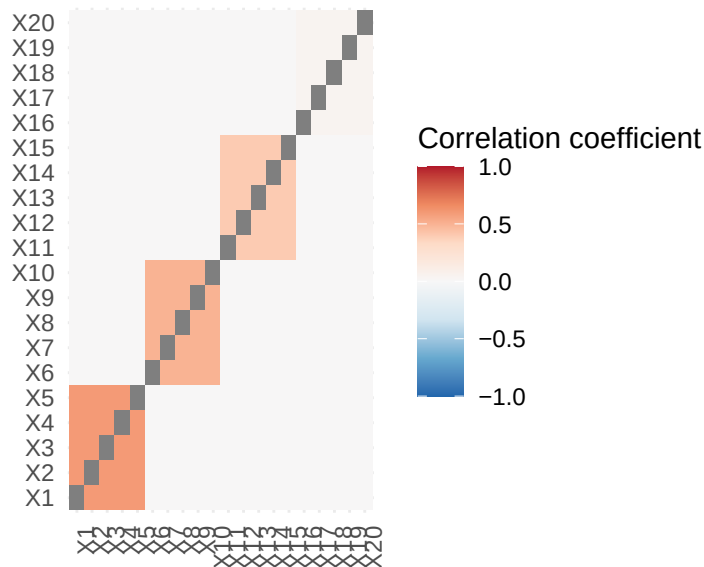
```

        block_size = 5,
        rho_within = c(0.6, 0.5, 0.4, 0.05),
        rho_between = 0.01)

# Inspect the matrix
colnames(Sigma) <- paste0("X",1:20)
rownames(Sigma) <- paste0("X",1:20)
Sigma %>%
  as.data.frame()%>%
  tibble::rownames_to_column("var1")%>%
  pivot_longer(-var1, names_to = "var2")%>%
  mutate(value = ifelse(var1==var2, NA, value),
         var1 = factor(var1, levels = paste0("X",1:20)),
         var2 = factor(var2, levels = paste0("X",1:20)))%>%
  {
    ggplot(., aes(var1, var2, fill = value)) +
      geom_tile() +
      scale_fill_distiller(palette = "RdBu", direction = -1,
                          limits = c(-1,1)) +
      theme_minimal() +
      theme(axis.text.x = element_text(angle = 90, hjust = 1)) +
      labs(fill = "Correlation coefficient", x = "", y = "",
           title = "True correlation matrix in the simulation design")
  }

```

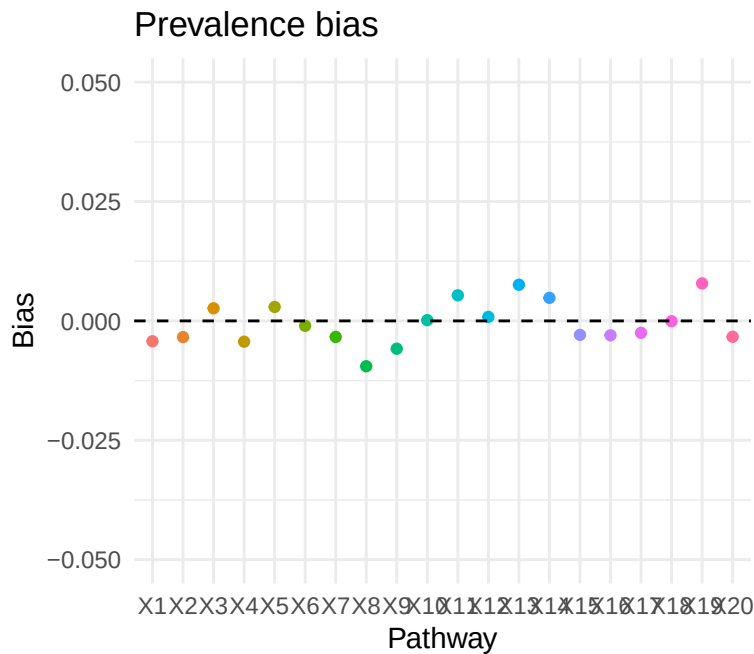
True correlation matrix in the simulation des



```
# Run simulations
dna_sims <- replicate(R,
  simulate_binary(n, prev, Sigma),
  simplify = FALSE) %>%
  imap(~ mutate(data.frame(.x), R = .y)) %>%
  bind_rows()

# Prevalence bias
dna_sims %>%
  bind_rows() %>%
  pivot_longer(-R, names_to = "pathway", values_to = "est_prev") %>%
  dplyr::full_join(rownames_to_column(data.frame(true_prev = prev),
    "pathway"), by = "pathway") %>%
  group_by(pathway) %>%
  summarize(bias = estimate_bias(true = true_prev, est = est_prev)) %>%
  mutate(pathway = factor(pathway, levels = paste0("X", 1:20))) %>%
  {
    ggplot(., aes(x = pathway, y = bias, colour = pathway)) +
      geom_point(aes(fill = "pathway")) +
      geom_hline(yintercept = 0, linetype = "dashed") +
      scale_y_continuous(limits = c(-0.05, 0.05)) +
      labs(x = "Pathway", y = "Bias", title = "Prevalence bias") +
      theme_minimal() +
  }
```

```
theme(legend.position = "none")
}
```



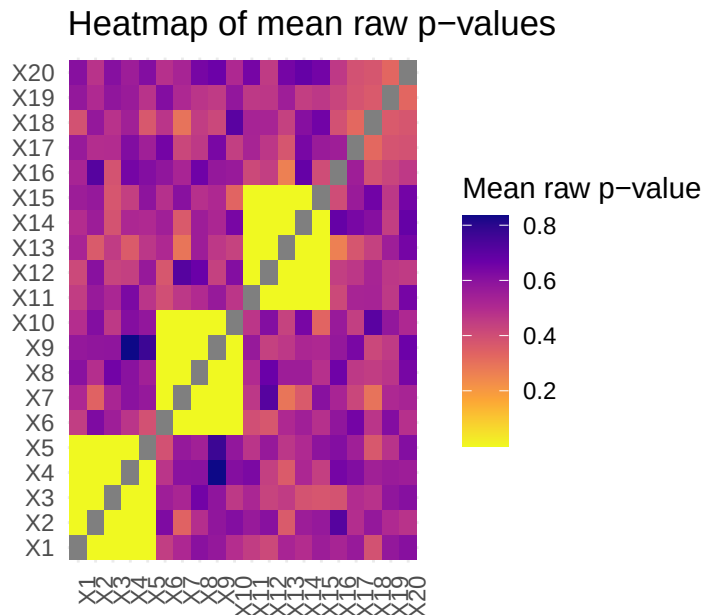
```
# Type I error
dna_sims_pval <- dna_sims %>%
  group_split(R) %>%
  map(~.x %>%
    dplyr::select(-R)%>%
    pairwise_fisher(vars = names()))

# Heatmap of mean raw p-values
dna_sims_pval %>%
  map(~as.data.frame(as.table(.x$p.value.matrix))%>%
    mutate(Freq = ifelse(Var1==Var2, NA, Freq))%>%
    reduce(bind_rows, .id = "R")%>%
    group_by(Var1, Var2)%>%
    summarise(mean_p.value = mean(Freq), .groups = "drop")%>%
  {
    ggplot(., aes(Var1, Var2, fill = mean_p.value)) +
      geom_tile() +
      scale_fill_viridis_c(option = "C", direction = -1) +
      theme_minimal() +
  }
```

```

theme(axis.text.x = element_text(angle = 90, hjust = 1)) +
labs(fill = "Mean row p-value", x = "", y = "",
      title = "Heatmap of mean row p-values")
}

```

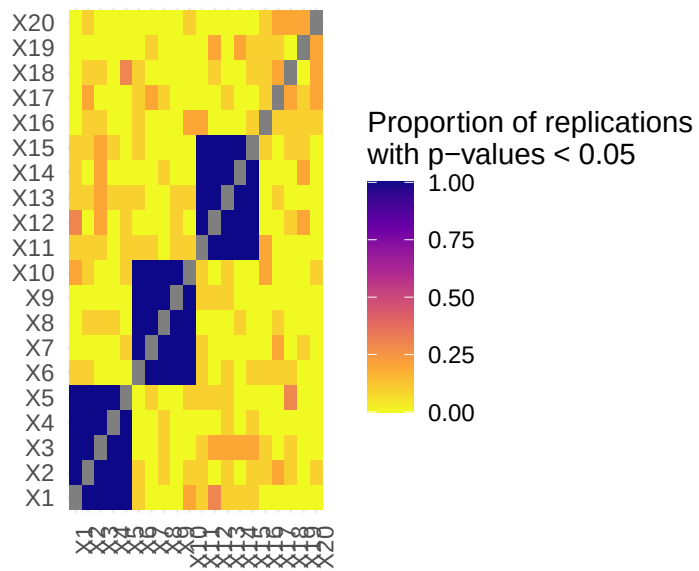


```

# Heatmap of the proportion of replications with p-values < 0.05
dna_sims_pval %>%
  map(~as.data.frame(as.table(.x$p.value.matrix))%>%
    mutate(Freq = ifelse(Var1==Var2, NA, Freq))%>%
  reduce(bind_rows, .id = "R")%>%
  group_by(Var1, Var2)%>%
  summarise(mean_typeI = mean(Freq < 0.05, na.rm = TRUE),
            .groups = "drop")%>%
  {
    ggplot(., aes(Var1, Var2, fill = mean_typeI)) +
      geom_tile() +
      scale_fill_viridis_c(option = "C", direction = -1) +
      theme_minimal() +
      theme(axis.text.x = element_text(angle = 90, hjust = 1)) +
      labs(fill = "Proportion of replications \nwith p-values < 0.05", x = "", y = "",
            title = "Heatmap of the proportion of replications with p-values < 0.05")
  }
}

```

Heatmap of the proportion of replications w



2 Simulate continuous variables

Example 1

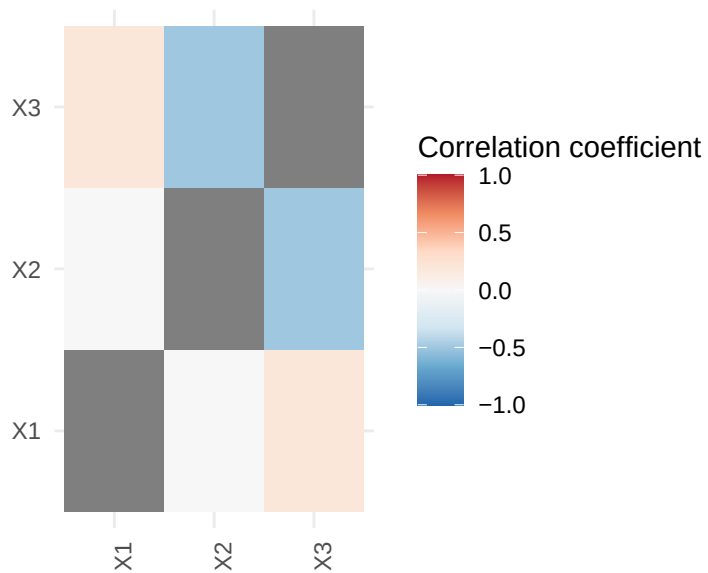
We generate a simple example.

- Sample size: $n = 1000$
- Number of RNA pathways: $p = 3$
- Correlation matrix:

$$\Sigma = \begin{pmatrix} 1 & 0 & 0.2 \\ 0 & 1 & 0.5 \\ 0.2 & 0.5 & 1 \end{pmatrix}$$

```
p <- 3
mu <- 0
Sigma <- matrix(c(1, 0, 0.2,
                  0, 1, -0.5,
                  0.2, -0.5, 1),
                3, 3)
# Inspect the matrix
colnames(Sigma) <- paste0("X",1:p)
rownames(Sigma) <- paste0("X",1:p)
Sigma %>%
  as.data.frame()%>%
  tibble::rownames_to_column("var1")%>%
  pivot_longer(-var1, names_to = "var2")%>%
  mutate(value = ifelse(var1==var2, NA, value),
         var1 = factor(var1, levels = paste0("X",1:p)),
         var2 = factor(var2, levels = paste0("X",1:p)))%>%
  {
    ggplot(., aes(var1, var2, fill = value)) +
      geom_tile() +
      scale_fill_distiller(palette = "RdBu", direction = -1,
                          limits = c(-1,1)) +
      theme_minimal() +
      theme(axis.text.x = element_text(angle = 90, hjust = 1)) +
      labs(fill = "Correlation coefficient", x = "", y = "",
           title = "True correlation matrix in the simulation design")
  }
```


True correlation matrix in the simulation desi



```
rna1 <- simulate_continuous(n, p, mu, Sigma)
```

```
# Check mean
colMeans(rna1)
```

```

              X1                      X2                      X3
0.00000000000000001686845 0.00000000000000001300696 0.00000000000000001945319

```

```
# Check variance
apply(rna1, 2, var)
```

```

X1 X2 X3
1  1  1

```

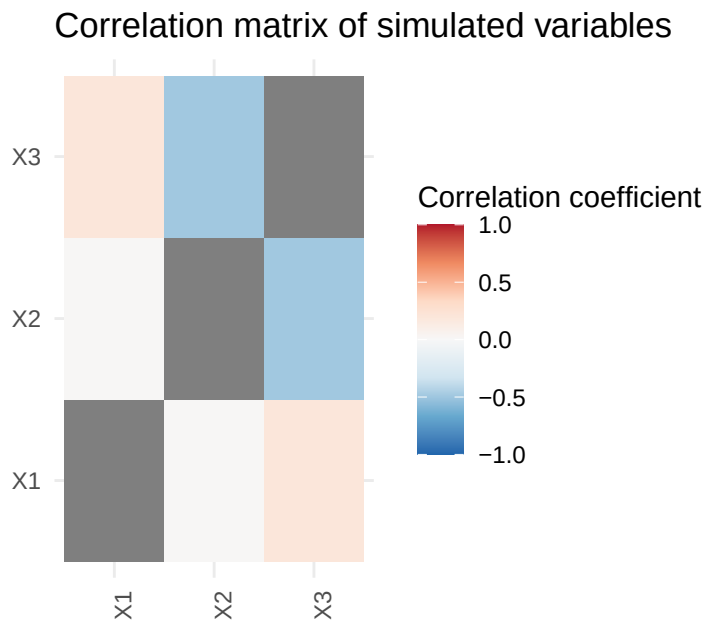
```
# Correlation matrix
as.data.frame(cor(rna1))%>%
  tibble::rownames_to_column("var1")%>%
  pivot_longer(-var1, names_to = "var2")%>%
  mutate(value = ifelse(var1==var2, NA, value),
         var1 = factor(var1, levels = paste0("X",1:20)),
         var2 = factor(var2, levels = paste0("X",1:20)))%>%

```

```

{
  ggplot(., aes(var1, var2, fill = value)) +
    geom_tile() +
    scale_fill_distiller(palette = "RdBu", direction = -1,
                        limits = c(-1,1)) +
    theme_minimal() +
    theme(axis.text.x = element_text(angle = 90, hjust = 1)) +
    labs(fill = "Correlation coefficient", x = "", y = "",
         title = "Correlation matrix of simulated variables")
}

```



Example 2

We simulate correlated variables grouped into blocks.

- Sample size: $n = 1000$
- Number of RNA pathways $p = 20$
- Block size: $b = 5$
- Number of blocks: $K = 4$
- Within-block correlations: $\rho_{k1} = 0.7$, $\rho_{k2} = -0.2$, $\rho_{k3} = 0.5$, $\rho_{k4} = 0.1$

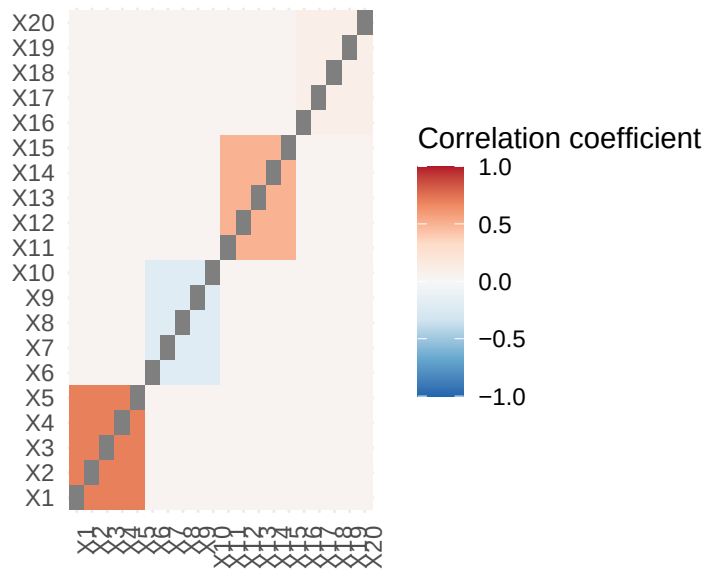
- Between-block correlation: $\rho_B = 0.05$

```
p = 20

# Initialize the matrix
Sigma <- generate_block_corr(p,
                             block_size = 5,
                             rho_within = c(0.7, -0.2, 0.5, 0.1),
                             rho_between = 0.05)

# Inspect the matrix
colnames(Sigma) <- paste0("X",1:20)
rownames(Sigma) <- paste0("X",1:20)
Sigma %>%
  as.data.frame()%>%
  tibble::rownames_to_column("var1")%>%
  pivot_longer(-var1, names_to = "var2")%>%
  mutate(value = ifelse(var1==var2, NA, value),
         var1 = factor(var1, levels = paste0("X",1:20)),
         var2 = factor(var2, levels = paste0("X",1:20)))%>%
  {
    ggplot(., aes(var1, var2, fill = value)) +
      geom_tile() +
      scale_fill_distiller(palette = "RdBu", direction = -1,
                           limits = c(-1,1)) +
      theme_minimal() +
      theme(axis.text.x = element_text(angle = 90, hjust = 1)) +
      labs(fill = "Correlation coefficient", x = "", y = "",
           title = "True correlation matrix in the simulation design")
  }
```

True correlation matrix in the simulation des



```
# Simulate correlated data using this matrix
rna_block <- simulate_continuous(n, p, mu, Sigma)

# Check mean
colMeans(rna_block)
```

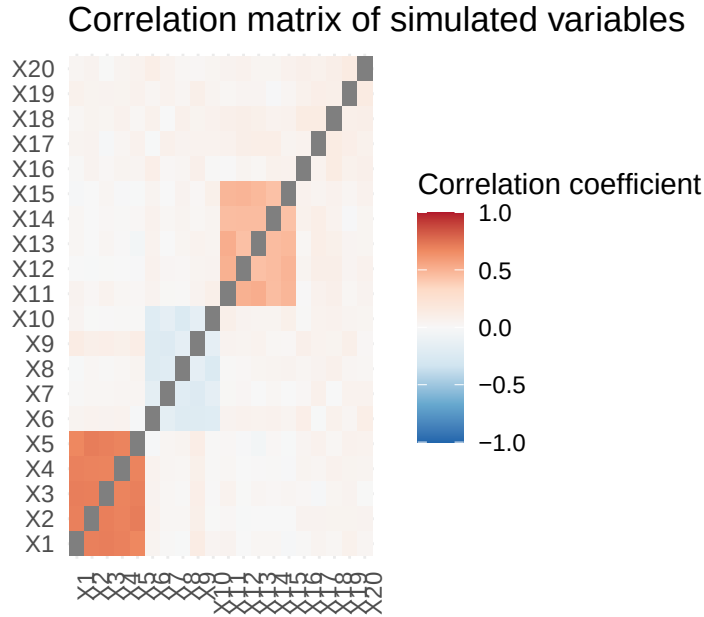
	X1	X2
	0.00000000000000006865602	-0.000000000000000017580555
	X3	X4
	0.00000000000000001023487	-0.000000000000000007291043
	X5	X6
	-0.000000000000000015929098	0.000000000000000006966649
	X7	X8
	0.000000000000000008755149	-0.0000000000000000023204962
	X9	X10
	0.0000000000000000021183142	-0.000000000000000009753483
	X11	X12
	-0.0000000000000000032161773	0.000000000000000003148523
	X13	X14
	0.0000000000000000017845100	-0.000000000000000005017688
	X15	X16
	-0.0000000000000000012673890	-0.0000000000000000027408631

	X17	X18
	-0.00000000000000002680148	0.000000000000000012176024
	X19	X20
	0.000000000000000017374557	-0.000000000000000033576440

```
# Check variance
apply(rna_block, 2, var)
```

X1	X2	X3	X4	X5	X6	X7	X8	X9	X10	X11	X12	X13	X14	X15	X16	X17	X18	X19	X20
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1

```
# Correlation matrix
as.data.frame(cor(rna_block))%>%
  tibble::rownames_to_column("var1")%>%
  pivot_longer(-var1, names_to = "var2")%>%
  mutate(value = ifelse(var1==var2, NA, value),
         var1 = factor(var1, levels = paste0("X",1:20)),
         var2 = factor(var2, levels = paste0("X",1:20)))%>%
  {
    ggplot(., aes(var1, var2, fill = value)) +
      geom_tile() +
      scale_fill_distiller(palette = "RdBu", direction = -1,
                          limits = c(-1,1)) +
      theme_minimal() +
      theme(axis.text.x = element_text(angle = 90, hjust = 1)) +
      labs(fill = "Correlation coefficient", x = "", y = "",
           title = "Correlation matrix of simulated variables")
  }
```



All simulations were repeated across $R = 1000$ replications.

For each pathway, we computed the empirical mean, variance, and pairwise correlation coefficients of the RNA pathways across simulation replicates. Mean, variance and correlation coefficients were averaged across replicates.

- Sample size: $n = 1000$
- Number of RNA pathways $p = 20$
- Block size: $b = 5$
- Number of blocks: $K = 4$
- Within-block correlations: $\rho_{k1} = 0.7$, $\rho_{k2} = -0.2$, $\rho_{k3} = 0.5$, $\rho_{k4} = 0.3$
- Between-block correlation: $\rho_B = 0.1$

```
p <- 20
mu <- 0

# Initialize the matrix
Sigma <- generate_block_corr(p,
                             block_size = 5,
                             rho_within = c(0.7, -0.2, 0.5, 0.3),
                             rho_between = 0.1)
```

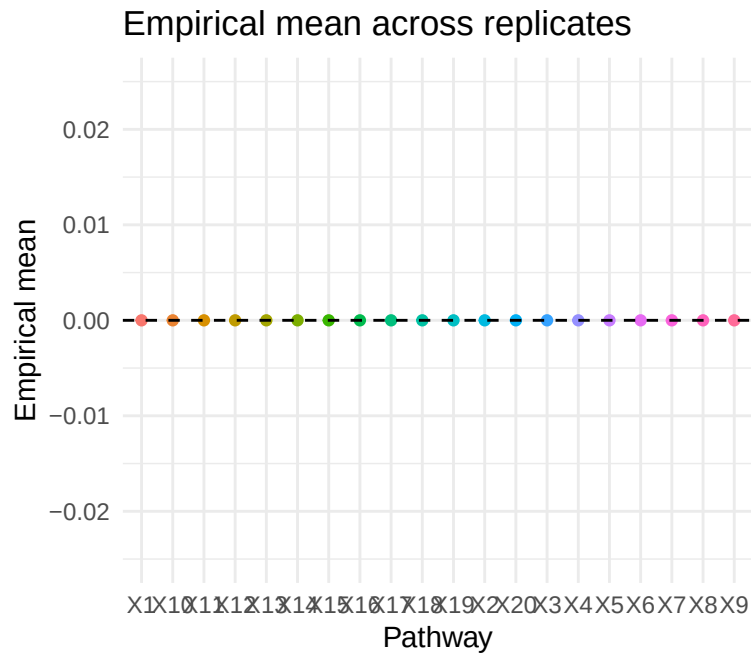
```

colnames(Sigma) <- paste0("X",1:20)
rownames(Sigma) <- paste0("X",1:20)

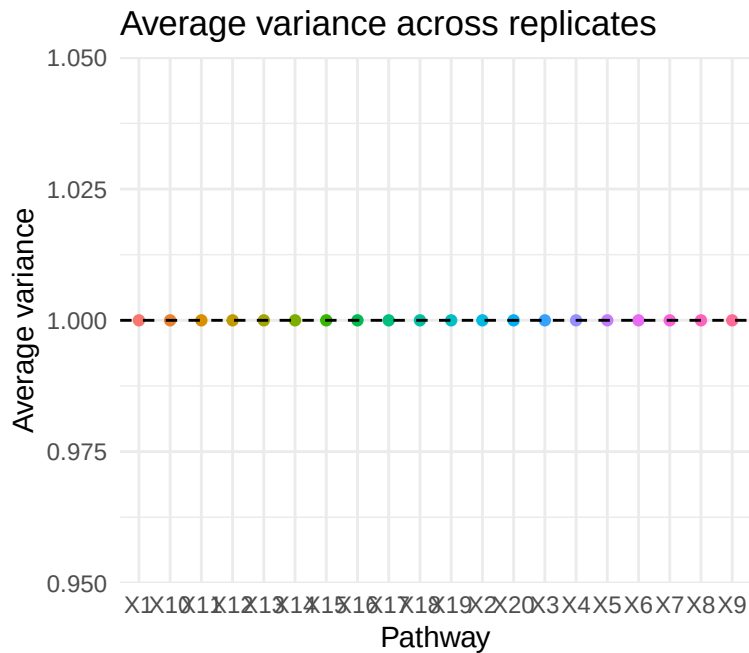
# Run simulations
rna_sims <- replicate(R,
                      simulate_continuous(n, p, mu, Sigma),
                      simplify = FALSE)

# Check mean
rna_sims %>%
  map(~colMeans(.x))%>%
  reduce(bind_rows)%>%
  summarise_all(mean)%>%
  pivot_longer(everything())%>%
  {
    ggplot(., aes(x = name, y = value, colour = name))+
      geom_point()+
      geom_hline(yintercept = 0, linetype = "dashed")+
      scale_y_continuous(limits = c(-0.025, 0.025))+
      labs(x = "Pathway", y = "Empirical mean",
           title = "Empirical mean across replicates")+
      theme_minimal()+
      theme(legend.position = "none")
  }

```

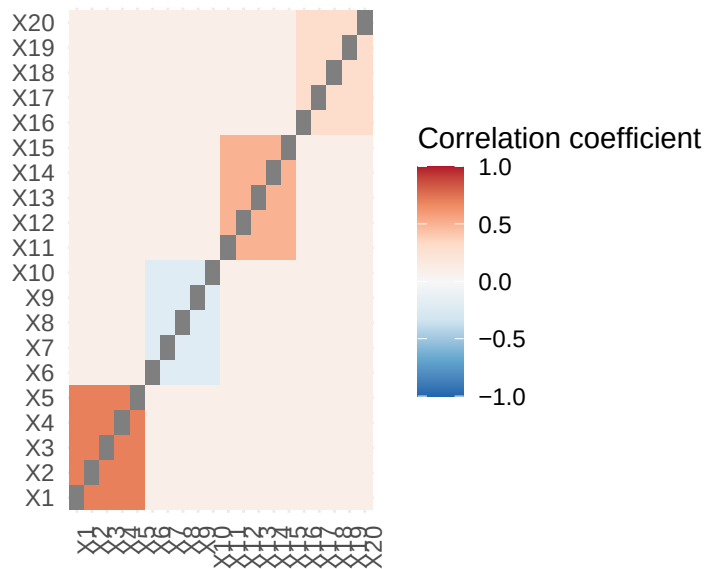


```
# Check variance
rna_sims %>%
  map(~ apply(., 2, var))%>%
  reduce(bind_rows)%>%
  summarise_all(mean)%>%
  pivot_longer(everything())%>%
  {
    ggplot(., aes(x = name, y = value, colour = name))+
      geom_point()+
      geom_hline(yintercept = 1, linetype = "dashed")+
      labs(x = "Pathway", y = "Average variance",
           title = "Average variance across replicates")+
      theme_minimal()+
      theme(legend.position = "none")
  }
```

```
# True correlation matrix
Sigma %>%
  as.data.frame()%>%
  tibble::rownames_to_column("var1")%>%
  pivot_longer(-var1, names_to = "var2")%>%
  mutate(value = ifelse(var1==var2, NA, value),
         var1 = factor(var1, levels = paste0("X",1:20)),
         var2 = factor(var2, levels = paste0("X",1:20)))%>%
  {
    ggplot(., aes(var1, var2, fill = value)) +
      geom_tile() +
      scale_fill_distiller(palette = "RdBu", direction = -1,
                          limits = c(-1,1)) +
      theme_minimal() +
      theme(axis.text.x = element_text(angle = 90, hjust = 1)) +
      labs(fill = "Correlation coefficient", x = "", y = "",
           title = "True correlation matrix in the simulation design")
  }
```

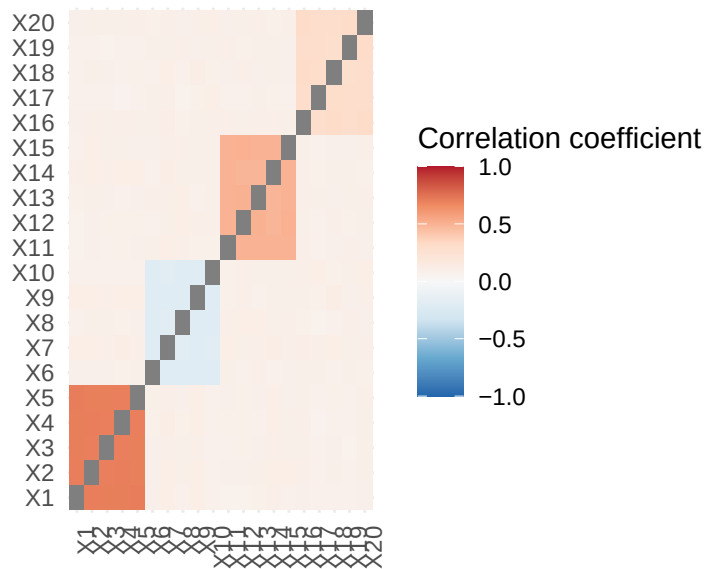
True correlation matrix in the simulation des



```
# Average correlation matrix across simulation replicates
cor_sims_rna <- rna_sims %>%
  map(cor) %>%
  reduce(`+`) / length(rna_sims)

as.data.frame(cor_sims_rna)%>%
  tibble::rownames_to_column("var1")%>%
  pivot_longer(-var1, names_to = "var2")%>%
  mutate(value = ifelse(var1==var2, NA, value),
         var1 = factor(var1, levels = paste0("X",1:20)),
         var2 = factor(var2, levels = paste0("X",1:20)))%>%
  {
    ggplot(., aes(var1, var2, fill = value)) +
      geom_tile() +
      scale_fill_distiller(palette = "RdBu", direction = -1,
                          limits = c(-1,1)) +
      theme_minimal() +
      theme(axis.text.x = element_text(angle = 90, hjust = 1)) +
      labs(fill = "Correlation coefficient", x = "", y = "",
           title = "Average correlation matrix across simulation replicates")
  }
```

Average correlation matrix across simulation



3 Simulate time-to-event data

3.1 Aalen additive model

Four binary covariates were generated *independently* with prevalence of 50%, $X_j \sim \mathcal{B}(0.4)$.

Events times were simulated under the following time-constant additive hazards model:

$$\lambda(t|x) = \lambda_0 + \alpha_1 X_1 + \alpha_2 X_2 + \alpha_3 X_3 + \alpha_4 X_4$$

with $\lambda_0 = 0.05$ and $\beta = (0.1, 0.05, 0.005, 0)$.

This means:

- Having $X_1 = 1$ increases the hazard by 0.1 compared with $X_1 = 0$
- Having $X_2 = 1$ increases the hazard by 0.05 compared with $X_2 = 0$
- Having $X_3 = 1$ increases the hazard by 0.005 compared with $X_3 = 0$
- There are no difference in hazard between $X_4 = 1$ and $X_4 = 0$

Interpretation:

- Each coefficient represents an absolute increase in instantaneous event rate
- Effects combine by addition
- A coefficient equal to zero means the variable does not contribute to the hazard

Independent censoring times were generated as $C \sim \text{Uniform}(0, c_{\max})$. The parameter $c_{\max}$ was calibrated using a large Monte Carlo sample from the data-generating mechanism to achieve an expected censoring proportion of approximately 5%, and the calibrated value was then fixed across all simulation replicates.

```
# Generate data
n <- 1000
alpha <- c(0.1, 0.05, 0.03, 0.01, 0.005, 0)
p <- length(alpha)
prev <- rep(0.4, p)
sigma <- diag(x = 1, nrow = p, ncol = p)
lambda0 <- 0.05
target_cens <- 0.05
c_max <- calibrate_c_max_aalen(lambda0      = lambda0,
                               alpha       = alpha,
                               prev        = prev,
                               sigma       = sigma,
                               target_cens = target_cens,
                               seed       = 2026)

c_max
```

```
[1] 195.9618
```

```
data_aalen <- simulate_aalen_data(n, prev, sigma, lambda0, alpha, c_max)
head(data_aalen)
```

	time	event	X1	X2	X3	X4	X5	X6
1	10.08510404	1	0	1	1	0	0	0
2	0.04453134	1	1	1	1	1	0	0
3	1.56105232	1	0	1	0	0	1	0
4	76.37566184	1	0	0	0	1	0	1
5	4.14109147	0	1	0	1	0	0	1
6	6.44473955	1	0	1	0	1	0	1

```
# Check prevalences
round(colMeans(data_aalen %>% dplyr::select(starts_with("X"))), 3)
```

	X1	X2	X3	X4	X5	X6
	0.416	0.394	0.393	0.383	0.393	0.413

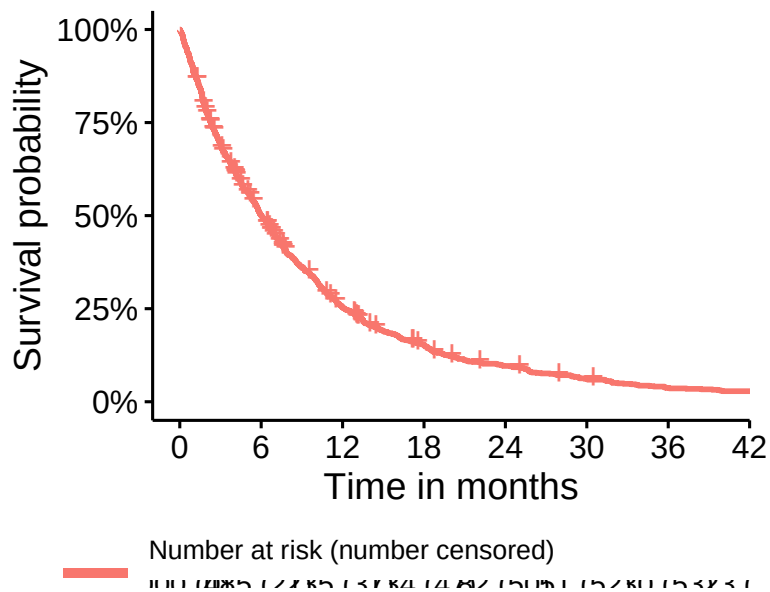
```
# Check censoring rate
mean(1 - data_aalen$event)
```

```
[1] 0.053
```

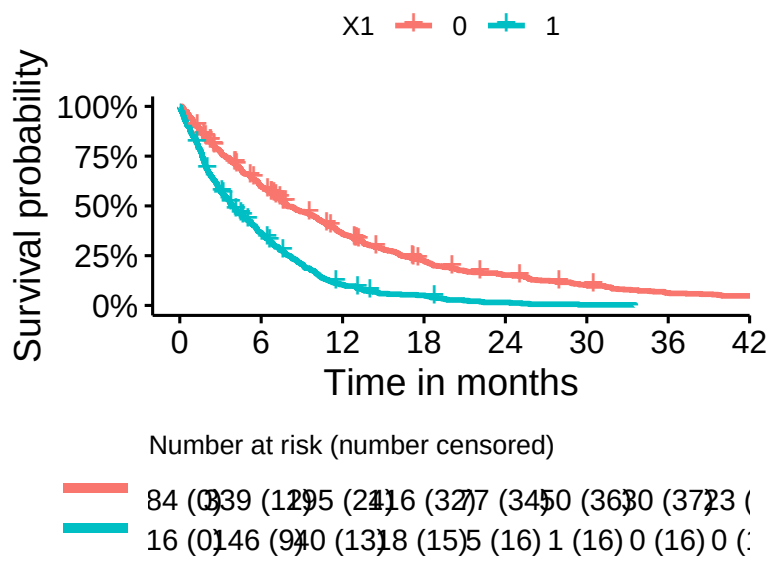
Kaplan-Meier

```
list_form <- c("1",paste0("X",1:p))%>%
  purrr::map(~as.formula(paste("Surv(time,event)~", .x)))
plot_surv(formula = list_form,
  data = data_aalen,
  legend.title = c("", paste0("X",1:p)))
```

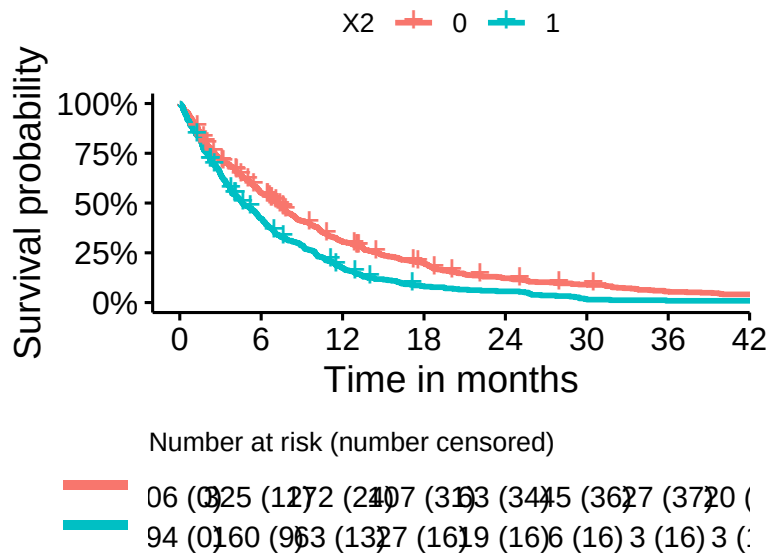
```
$data_null_model
```



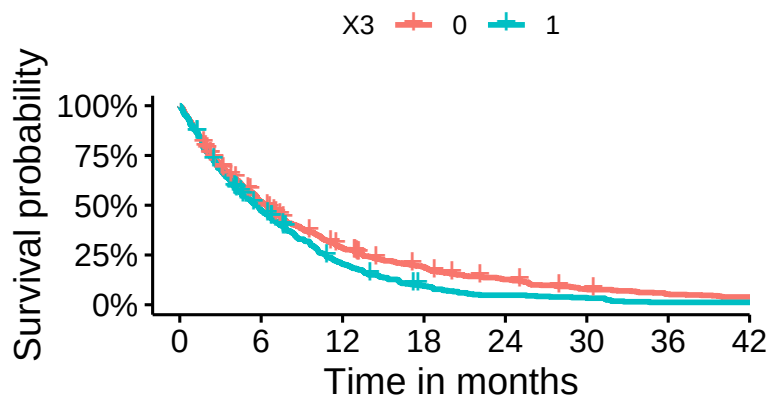
\$data_X1



\$data_X2



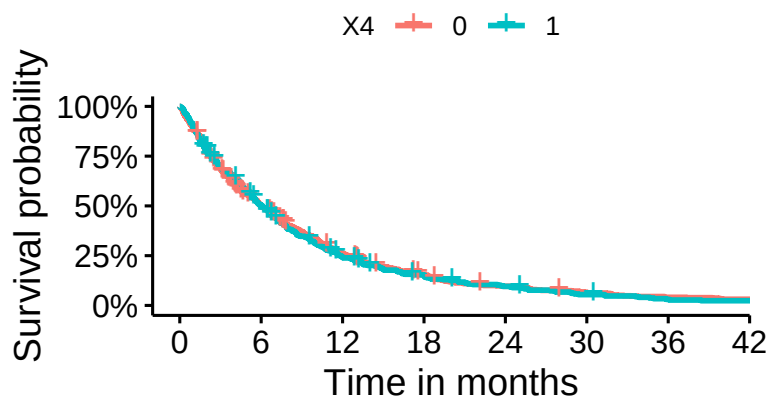
\$data_X3



Number at risk (number censored)

Time (months)	0	6	12	18	24	30	36	42
X3=0 (red)	07 (0)	06 (12)	05 (21)	03 (28)	06 (31)	09 (33)	06 (34)	09 (34)
X3=1 (teal)	93 (0)	79 (9)	74 (16)	61 (19)	46 (19)	42 (19)	4 (19)	4 (19)

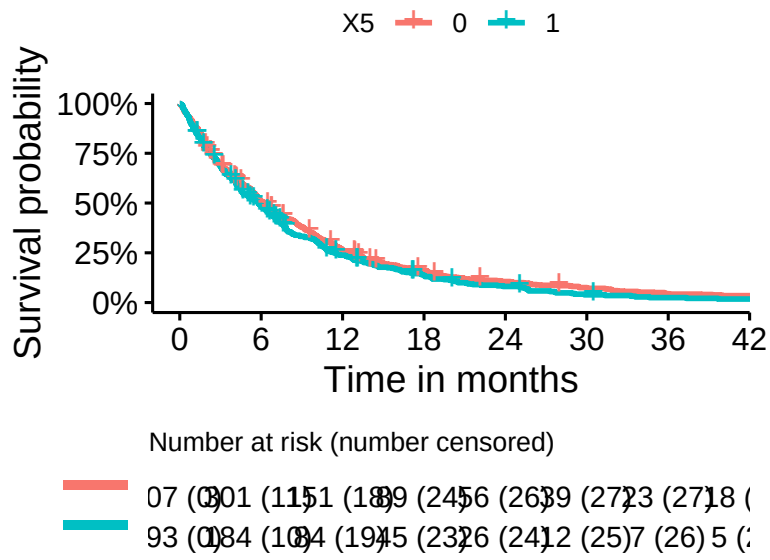
\$data_X4



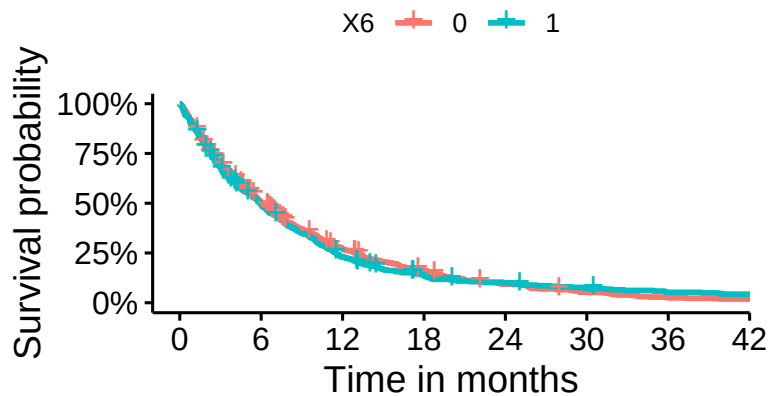
Number at risk (number censored)

Time (months)	0	6	12	18	24	30	36	42
X4=0 (red)	17 (0)	08 (13)	05 (23)	04 (28)	05 (30)	04 (31)	01 (31)	06 (31)
X4=1 (teal)	83 (0)	87 (8)	85 (15)	50 (19)	31 (20)	17 (21)	09 (22)	7 (22)

\$data_X5



\$data_X6



Number at risk (number censored)

—	87 (0) 88 (11) 45 (25) 34 (29) 47 (31) 26 (32) 13 (32) 9 (32)
—	13 (0) 97 (10) 90 (12) 50 (18) 35 (19) 25 (20) 17 (21) 14 (21)

Aalen model

```
# Fit the model
aalen_fit <- timereg::aalen(Surv(time, event)~X1+X2+X3+X4, data = data_aalen)
summary(aalen_fit)
```

Additive Aalen Model

Test for nonparametric terms

Test for non-significant effects

	Supremum-test of significance	p-value	H ₀ : B(t)=0
(Intercept)	9.07	0.000	
X1	8.55	0.000	
X2	5.46	0.000	
X3	4.35	0.001	
X4	2.04	0.595	

Test for time invariant effects

	Kolmogorov-Smirnov test	p-value	H ₀ : constant effect
(Intercept)	1.230	0.000	
X1	3.060	0.000	
X2	1.530	0.000	

X3	1.110	0.000
X4	0.292	0.513
Cramer von Mises test p-value H_0:constant effect		
(Intercept)	55.000	0.00
X1	285.000	0.00
X2	63.800	0.00
X3	35.800	0.00
X4	0.538	0.74

Call:
timereg::aalen(formula = Surv(time, event) ~ X1 + X2 + X3 + X4,
data = data_aalen)

Test for non-significant effects

The model estimates the cumulative effects

$$A_j(t) = \int_0^t \alpha_j(u) du.$$

The reported *Supremum test* evaluates $H_0 : A_j(t) = 0$, meaning the covariate has no effect on the hazard over time.

Interpretation of the results

- Intercept: $p < 0.001 \rightarrow$ Baseline hazard is clearly non zero
- X_1 : $p < 0.001 \rightarrow$ Strong evidence that X_1 affects the hazard
- X_2 : $p < 0.001 \rightarrow$ Strong evidence that X_2 affects the hazard
- X_3 : $p = 0.001 \rightarrow$ No evidence that X_3 affects the hazard
- X_4 : $p = 0.6 \rightarrow$ No evidence that X_4 affects the hazard

Test for time invariant effects

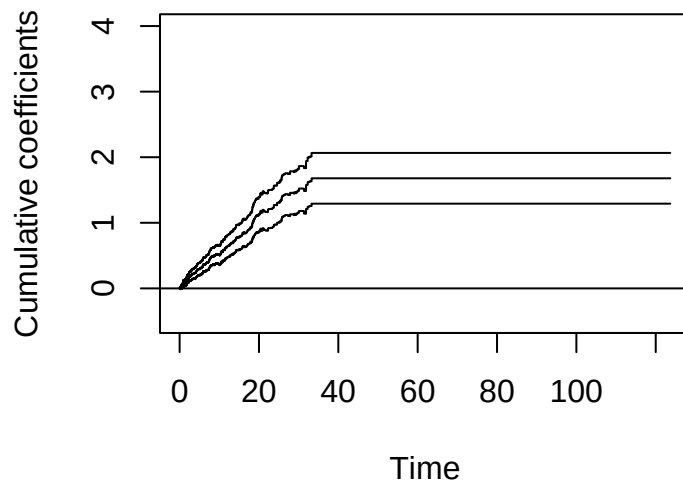
Interpretation of the results of the Kolmogorov–Smirnov test

Small p-values *rigtharrow* evidence of time-varying effects.

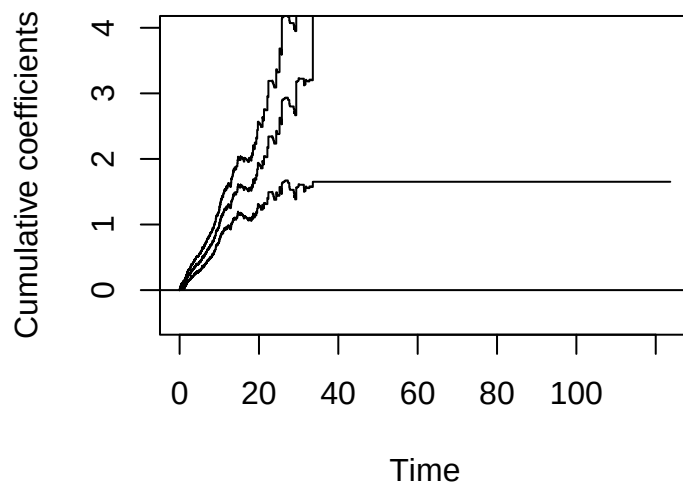
- Intercept: $p < 0.001 \rightarrow$ Baseline hazard clearly varies over time
- X_1 : $p < 0.001 \rightarrow$ Strong evidence that effect of X_1 changes over time
- X_2 : $p < 0.001 \rightarrow$ Strong evidence that effect of X_2 changes over time
- X_3 : $p < 0.001 \rightarrow$ No strong evidence against constant effect
- X_4 : $p = 0.5 \rightarrow$ No evidence of time variation

```
# Plot the non-parametric cumulative regression coefficients with 95% CI  
plot(aalen_fit, ylim = c(-0.5,4))
```

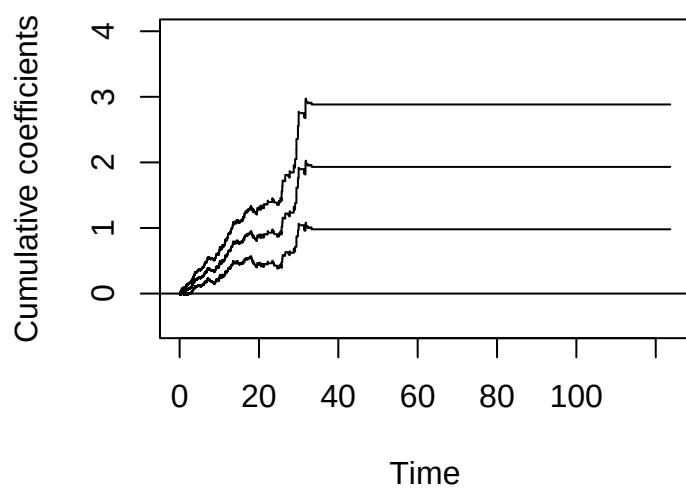
(Intercept)



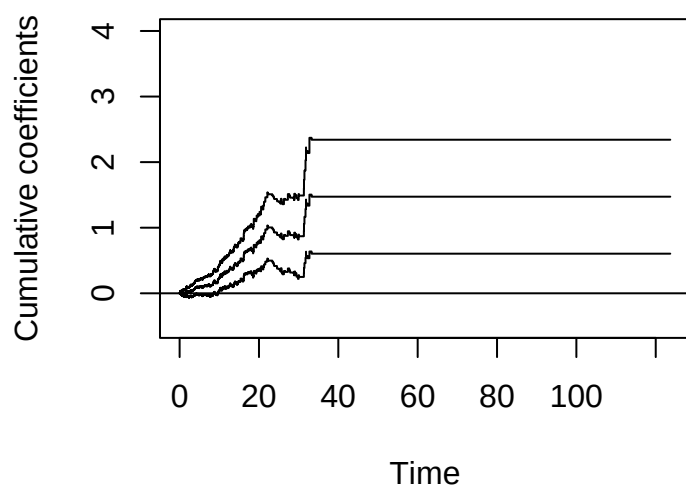
X1

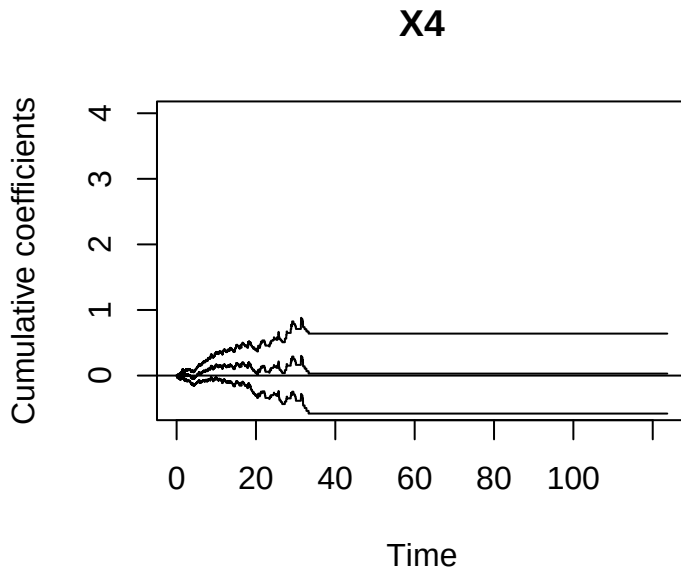


X2



X3





We simulated $R = 1000$ datasets.

```
test_aalen <- replicate(R, {
  # Generate data
  data <- simulate_aalen_data(n, prev, sigma, lambda0, alpha, c_max)
  # Estimate prevalences
  prev <- colMeans(data %>% dplyr::select(starts_with("X")))%>%
    stats::setNames(paste0("prev_", names(.)))
  # Censoring rate
  cens_rate <- stats::setNames(mean(1 - data$event), "cens_rate")
  # Fit Aalen model
  fit_obj <- timereg::aalen(Surv(time, event)~X1+X2+X3+X4, data)
  # Tests
  pval_supremum <- fit_obj[["pval.testBeq0"]]
  names(pval_supremum) <- paste0("pval_supremum_", names(pval_supremum))
  pval_KS <- fit_obj[["pval.testBeqC"]]
  names(pval_KS) <- paste0("pval_KS_", names(pval_KS))
  pval_CvM <- fit_obj[["pval.testBeqC.is"]]
  names(pval_CvM) <- paste0("pval_CvM_", names(pval_CvM))
})
```

```

c(prev, cens_rate, pval_supremum, pval_KS, pval_CvM)
}, simplify = F)%>%
bind_rows

data.frame(colMeans(test_aalen))%>%
  tibble::rownames_to_column()%>%
  kable(col.names = c("Name", "Value"))

```

Name	Value
prev_X1	0.4082
prev_X2	0.4033
prev_X3	0.3998
prev_X4	0.3998
prev_X5	0.4006
prev_X6	0.4013
cens_rate	0.0496
pval_supremum_(Intercept)	0.0000
pval_supremum_X1	0.0000
pval_supremum_X2	0.0000
pval_supremum_X3	0.0185
pval_supremum_X4	0.4244
pval_KS_(Intercept)	0.0000
pval_KS_X1	0.0000
pval_KS_X2	0.0004
pval_KS_X3	0.0200
pval_KS_X4	0.3882
pval_CvM_(Intercept)	0.0000
pval_CvM_X1	0.0000
pval_CvM_X2	0.0001
pval_CvM_X3	0.0158
pval_CvM_X4	0.3949

```

test_aalen %>%
  dplyr::select(starts_with("pval"))%>%
  dplyr::mutate(across(everything(), ~ifelse(.x<0.05, 1, 0)))%>%
  dplyr::summarise_all(mean)%>%
  tidyr::pivot_longer(everything())%>%
  kable(col.names = c("p-value", "Rejection probability"))

```

p-value	Rejection probability
pval_supremum_(Intercept)	1.0
pval_supremum_X1	1.0
pval_supremum_X2	1.0
pval_supremum_X3	0.9
pval_supremum_X4	0.0
pval_KS_(Intercept)	1.0
pval_KS_X1	1.0
pval_KS_X2	1.0
pval_KS_X3	0.8
pval_KS_X4	0.0
pval_CvM_(Intercept)	1.0
pval_CvM_X1	1.0
pval_CvM_X2	1.0
pval_CvM_X3	0.9
pval_CvM_X4	0.2

3.2 Cox-Aalen model

The covariates are generated *independently* as follows:

$$X = (X_1, X_2, X_3, Z_1, Z_2, Z_3)$$

where

- $X_1, X_2, X_3 \sim \mathcal{B}(0.4)$
- $Z_1, Z_2 \sim \mathcal{B}(0.4)$
- $Z_3 \sim \mathcal{N}(0, 1)$

To generate mixed additive and multiplicative effects, event times were simulated under a Cox-Aalen model with constant baseline hazard: $\lambda_0(t) = 0.05$. Additive effects were assigned to X_1 , X_2 and X_3 with coefficients $\alpha_1 = 0.1$, $\alpha_2 = 0.05$ and $\alpha_3 = 0$, respectively. Multiplicative (proportional hazards) effects were assigned to Z_1 , Z_2 and Z_3 with coefficients $\beta_1 = 1$, $\beta_2 = 0.5$ and $\beta_3 = 0$, corresponding to hazard ratios of approximately 2.72, 1.65 and 1.

Independent censoring times were generated as $C \sim \text{Uniform}(0, c_{max})$. The parameter c_{max} was calibrated using a large Monte Carlo sample from the data-generating mechanism to achieve an expected censoring proportion of approximately 5%, and the calibrated value was then fixed across all simulation replicates.


```

n <- 1000
prev_add <- c(0.4, 0.4, 0.4)
sigma_add <- diag(x = 1, nrow = 3, ncol = 3)
alpha <- c(0.1, 0.05, 0)
prev_mult <- c(0.4, 0.4)
sigma_mult_bin <- diag(x = 1, nrow = 2, ncol = 2)
mean_mult <- 0
sigma_mult_cont <- diag(x = 1, nrow = 1, ncol = 1)
beta <- c(1, 0.5, 0)
lambda0 <- 0.05
target_cens <- 0.05
c_max <- calibrate_c_max_cox_aalen(lambda0      = lambda0,
                                   alpha         = alpha,
                                   beta          = beta,
                                   prev_add      = prev_add,
                                   sigma_add     = sigma_add,
                                   prev_mult     = prev_mult,
                                   mean_mult     = mean_mult,
                                   sigma_mult_bin = sigma_mult_bin,
                                   sigma_mult_cont = sigma_mult_cont,
                                   target_cens    = target_cens,
                                   seed          = 2026)

c_max

```

```
[1] 151.3418
```

```

data_cox_aalen <- simulate_cox_aalen_data(
  n, lambda0, alpha, beta,
  prev_add, sigma_add, prev_mult, mean_mult,
  sigma_mult_bin, sigma_mult_cont, c_max
)
head(data_cox_aalen)

```

	time	event	X1	X2	X3	Z1	Z2	Z3
1	15.9040045	1	0	0	0	0	1	0.06569121
2	0.0432154	1	1	0	0	0	1	-0.29487658
3	1.6391049	1	0	1	0	0	0	0.19398223
4	30.5502647	1	1	0	1	0	0	1.33259324
5	3.1981744	0	0	0	1	1	1	-1.51629094
6	4.7261423	1	1	0	1	0	0	0.23890336

```
# Check prevalences
colMeans(data_cox_aalen %>% dplyr::select(X1:X3,Z1:Z2))
```

```
      X1      X2      X3      Z1      Z2
0.383 0.393 0.413 0.408 0.393
```

```
# Check mean and sd
round(mean(data_cox_aalen$Z3), 3)
```

```
[1] 0
```

```
sd(data_cox_aalen$Z3)
```

```
[1] 1
```

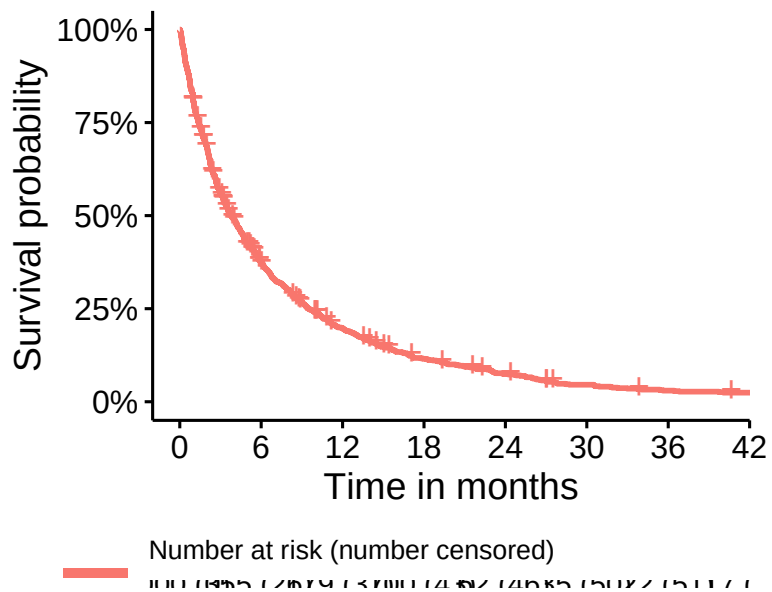
```
# Check censoring rate
mean(1 - data_cox_aalen$event)
```

```
[1] 0.055
```

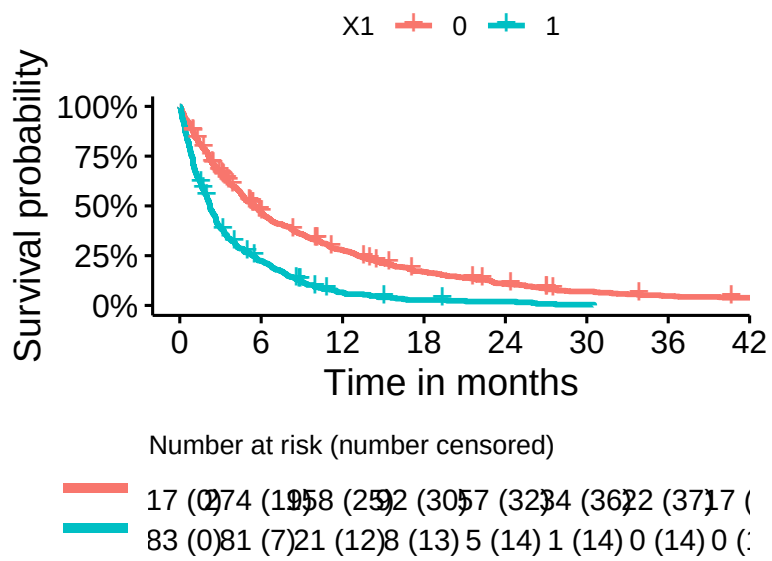
Kaplan-Meier

```
list_form <- c("1",paste0("X",1:3), paste0("Z",1:2))%>%
  purrr::map(~as.formula(paste("Surv(time,event)~", .x)))
plot_surv(formula = list_form,
          data = data_cox_aalen,
          legend.title = c("", paste0("X",1:3), paste0("Z",1:2)))
```

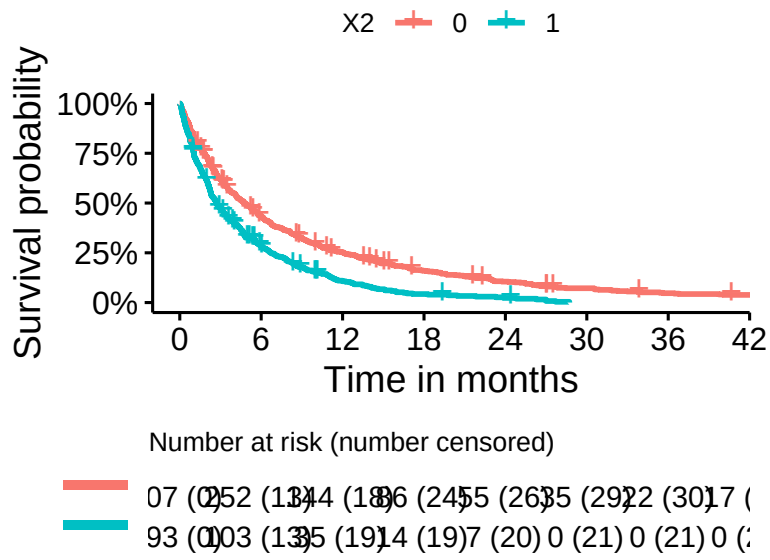
```
$data_null_model
```



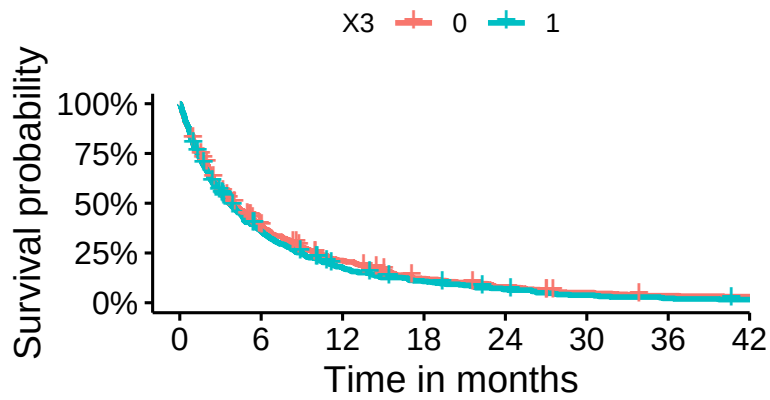
\$data_X1



\$data_X2



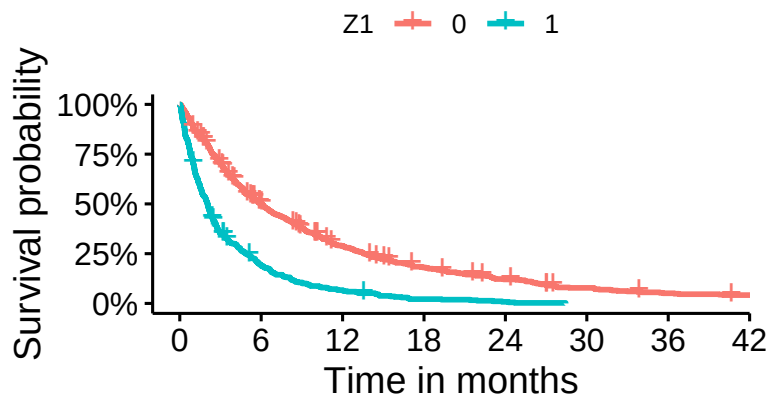
\$data_X3



Number at risk (number censored)

Time (months)	0	6	12	18	24	30	36	42
X3 = 0	87 (0)	14 (1)	14 (2)	12 (2)	10 (2)	7 (3)	5 (3)	3 (4)
X3 = 1	13 (0)	4 (1)	5 (1)	6 (3)	8 (1)	7 (2)	7 (2)	4 (3)

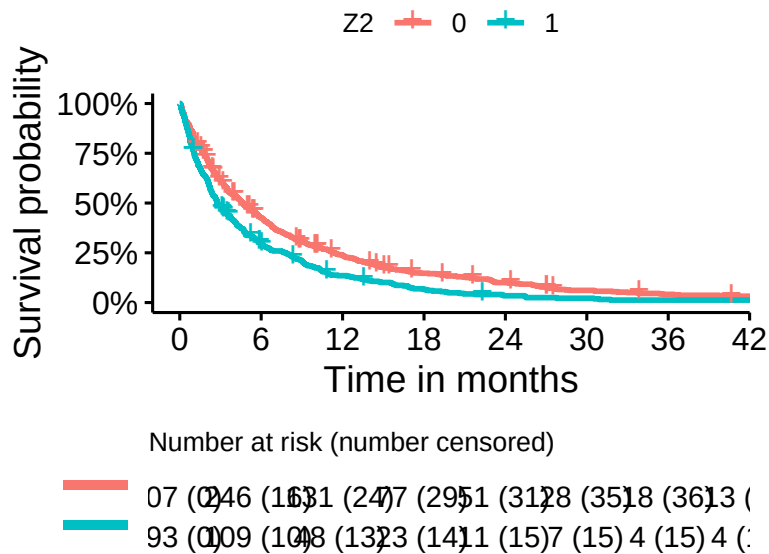
\$data_Z1



Number at risk (number censored)

Time (months)	0	6	12	18	24	30	36	42
Z1 = 0	92 (0)	81 (1)	54 (3)	42 (3)	35 (5)	35 (4)	22 (4)	17 (1)
Z1 = 1	08 (0)	74 (7)	25 (7)	8 (8)	3 (8)	0 (8)	0 (8)	0 (8)

\$data_Z2



Cox-Aalen model

```
# Fit Cox-Aalen model
cox_aalen_fit <- timereg::cox.aalen(
  Surv(time, event)~X1+X2+X3+prop(Z1)+prop(Z2)+prop(Z3),
  data = data_cox_aalen, n.sim = 1000, basesim = 1)
summary(cox_aalen_fit)
```

Cox-Aalen Model

Test for Aalen terms

Test for nonparametric terms

Test for non-significant effects

	Supremum-test of significance	p-value	H ₀ : B(t)=0
(Intercept)	10.20	0.000	
X1	8.13	0.000	
X2	5.67	0.000	
X3	1.34	0.937	

Test for time invariant effects

	Kolmogorov-Smirnov test	p-value	H ₀ :constant effect
(Intercept)	1.0800		0.000
X1	1.6100		0.000
X2	2.4900		0.000
X3	0.0967		0.946

Proportional Cox terms :

	Coef.	SE Robust	SE	D2log(L) ⁻¹	z	P-val
prop(Z1)	1.1200	0.0737	0.0706	0.0733	15.90	0.000000000000000000
prop(Z2)	0.5140	0.0713	0.0646	0.0695	7.95	0.000000000000000178
prop(Z3)	0.0276	0.0330	0.0310	0.0329	0.89	0.373000000000000000

lower2.5% upper97.5%

prop(Z1)	0.9760	1.2600
prop(Z2)	0.3740	0.6540
prop(Z3)	-0.0371	0.0923

Test of Proportionality

	sup hat U(t)	p-value	H ₀
prop(Z1)	11.8	0.296	
prop(Z2)	15.8	0.114	
prop(Z3)	24.2	0.398	

Test for non-significant effects

- Intercept: $p < 0.001 \rightarrow$ Baseline hazard is clearly non zero
- X_1 : $p < 0.001 \rightarrow$ Strong evidence that X_1 affects the hazard
- X_2 : $p < 0.001 \rightarrow$ No evidence that X_2 affects the hazard

Test for time invariant effects

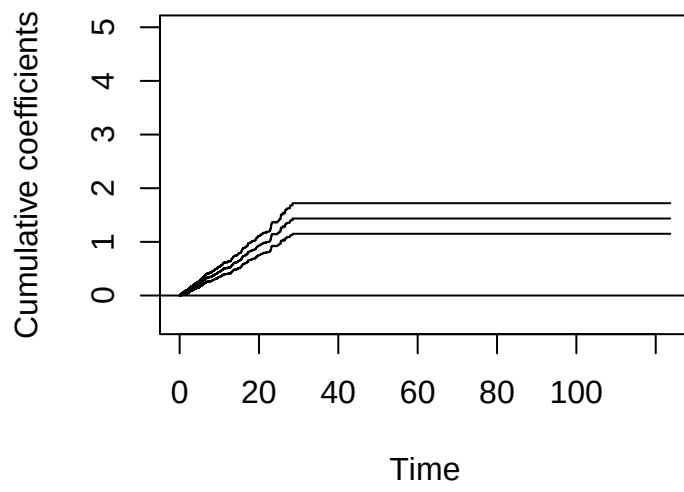
- Intercept: $p < 0.001 \rightarrow$ Baseline hazard clearly varies over time
- X_1 : $p < 0.001 \rightarrow$ Strong evidence that X_1 affects the hazard
- X_2 : $p < 0.001 \rightarrow$ No evidence of time variation

Test of proportionality

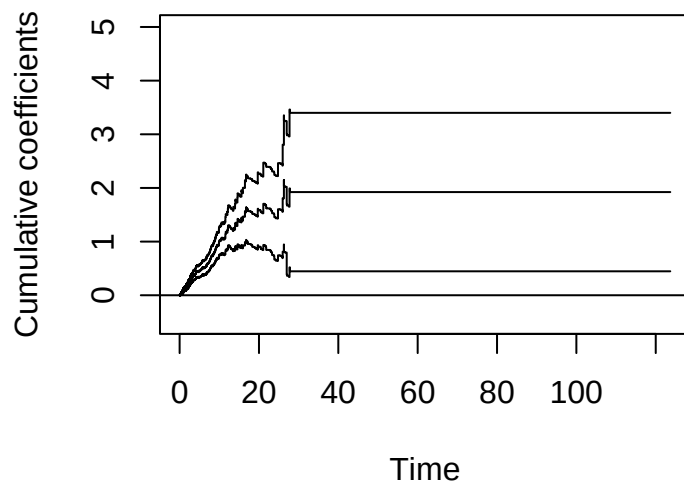
- X_3 : $p = 0.3 \rightarrow$ Compatible with a proportional effect
- X_4 : $p = 0.11 \rightarrow$ Compatible with a proportional effect

```
# Plot additive Aalen terms
plot(cox_aalen_fit, ylim = c(-0.5,5))
```

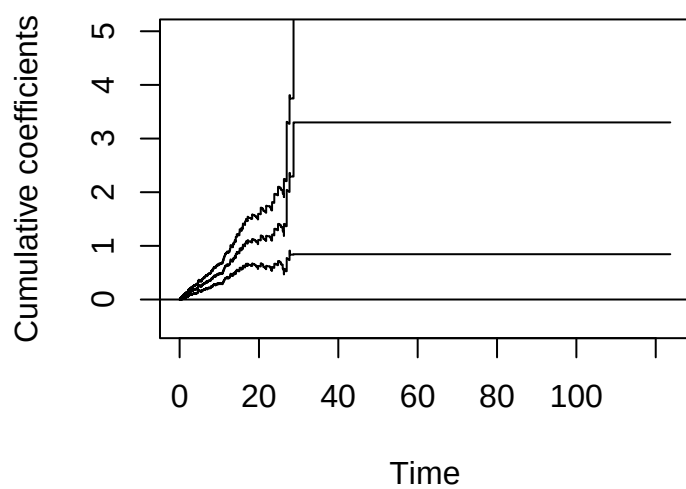
(Intercept)



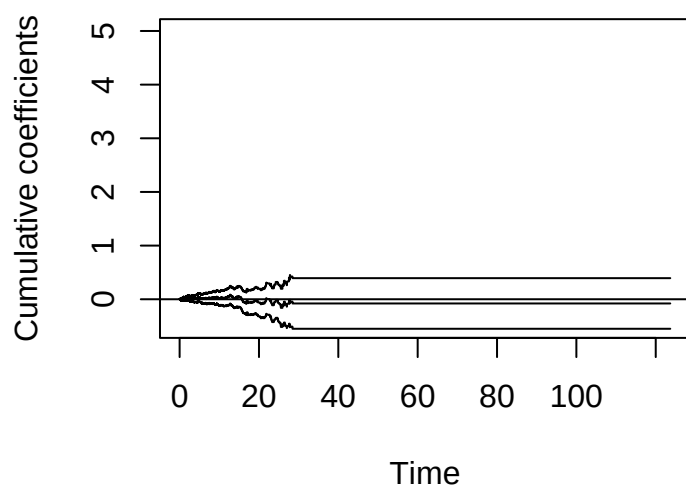
X1



X2

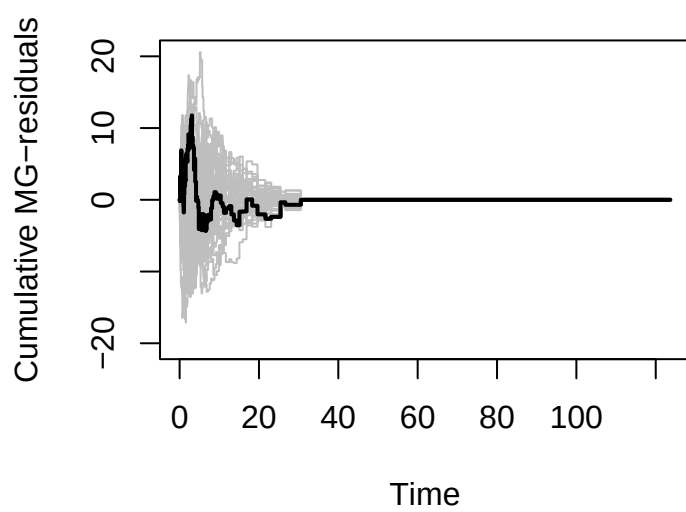


X3

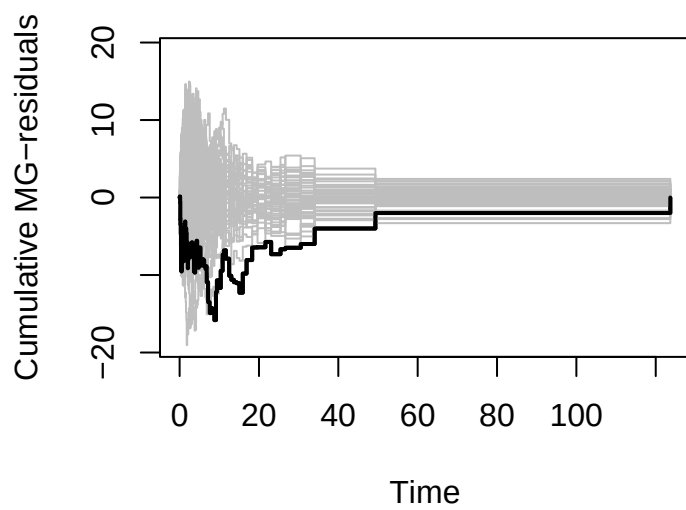


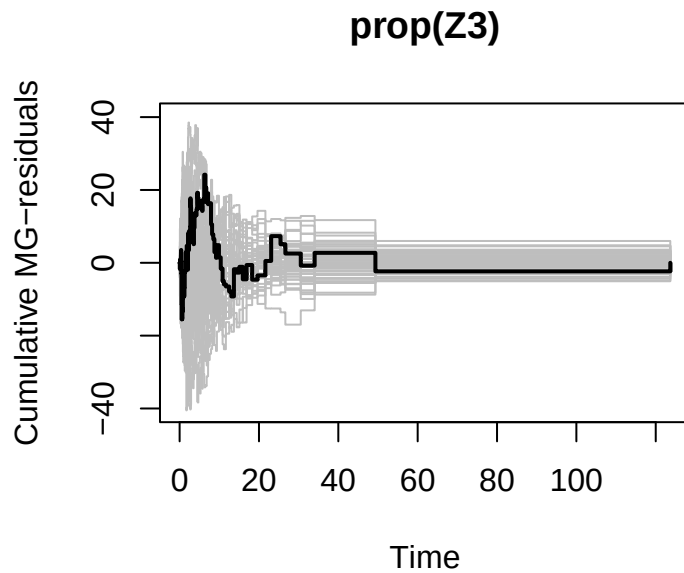
```
# Plot multiplicative terms  
plot(cox_aalen_fit, score= TRUE)
```

prop(Z1)



prop(Z2)





We simulated $R = 1000$ datasets.

```
test_cox_aalen <- replicate(R, {
  # Generate data
  data <- simulate_cox_aalen_data(n, lambda0, alpha, beta,
                                prev_add, sigma_add, prev_mult, mean_mult,
                                sigma_mult_bin, sigma_mult_cont, c_max)

  # Estimate prevalences
  prev <- colMeans(data %>% dplyr::select(X1:X3))%>%
    stats::setNames(paste0("prev_", names(.)))
  # Mean and sd Z3
  mean_sd_Z3 <- stats::setNames(c(mean(data$Z3), sd(data$Z3)),
                                c("mean_Z3", "sd_Z3"))

  # Censoring rate
  cens_rate <- stats::setNames(mean(1 - data$event), "cens_rate")
  # Fit Cox-Aalen model
  fit_obj <- timereg::cox.aalen(
    Surv(time, event)~X1+X2+X3+prop(Z1)+prop(Z2)+prop(Z3),
    data = data, n.sim = 1000, basesim = 1
  )
})
```

```

# Additive terms
pval_supremum <- fit_obj[["pval.testBeq0"]]
names(pval_supremum) <- paste0("pval_supremum_", names(pval_supremum))
pval_KS <- fit_obj[["pval.testBeqC"]]
names(pval_KS) <- paste0("pval_KS_", names(pval_KS))
# pval_CvM <- fit_obj[["pval.testBeqC.is"]]
# names(pval_CvM) <- paste0("pval_CvM", names(pval_CvM))
# Multiplicative terms
coef <- coef(fit_obj)[,"Coef."]
# Test for proportional
pval_prop <- stats::setNames(
  fit_obj$pval.Prop,
  gsub("prop\\(|\\)", "", paste0("pval_prop_", names(fit_obj$pval.Prop)))
)

c(prev, mean_sd_Z3, cens_rate,
  pval_supremum, pval_KS, #pval_CvM,
  coef, pval_prop)

}, simplify = F)%>%
bind_rows()

data.frame(round(colMeans(test_cox_aalen),3))%>%
tibble::rownames_to_column()%>%
kable(col.names = c("Name", "Value"))

```

Name	Value
prev_X1	0.395
prev_X2	0.402
prev_X3	0.399
mean_Z3	0.000
sd_Z3	1.000
cens_rate	0.046
pval_supremum_(Intercept)	0.000
pval_supremum_X1	0.000
pval_supremum_X2	0.000
pval_supremum_X3	0.574
pval_KS_(Intercept)	0.000
pval_KS_X1	0.000
pval_KS_X2	0.026
pval_KS_X3	0.359

Name	Value
prop(Z1)	0.974
prop(Z2)	0.554
prop(Z3)	0.002
pval_prop_Z1	0.662
pval_prop_Z2	0.369
pval_prop_Z3	0.486

```
test_cox_aalen %>%
  dplyr::select(starts_with("pval"))%>%
  dplyr::mutate(across(everything(), ~ifelse(.x<0.05, 1, 0)))%>%
  dplyr::summarise_all(mean)%>%
  tidyr::pivot_longer(everything())%>%
  mutate(value = round(value, 3))%>%
  kable(col.names = c("p-value", "Rejection probability"))
```

p-value	Rejection probability
pval_supremum_(Intercept)	1.0
pval_supremum_X1	1.0
pval_supremum_X2	1.0
pval_supremum_X3	0.0
pval_KS_(Intercept)	1.0
pval_KS_X1	1.0
pval_KS_X2	0.9
pval_KS_X3	0.0
pval_prop_Z1	0.0
pval_prop_Z2	0.2
pval_prop_Z3	0.2